

Nama : Jovita Fashya Islami

Nim : H1D024125

Shift Lama : G

Shift Baru : F

LAPORAN PRAKTIKUM KECERDASAN BUATAN MODUL 10

Program ini merupakan implementasi Algoritma Genetika (Genetic Algorithm) untuk menyelesaikan kasus Knapsack Problem, yaitu menentukan barang apa saja yang dipilih agar mendapatkan keuntungan maksimal tanpa melebihi kapasitas gudang.

Pada bagian awal program dilakukan import library:

```
import random
import matplotlib.pyplot as plt
```

Library random digunakan untuk menghasilkan angka acak pada proses algoritma genetika, sedangkan matplotlib.pyplot digunakan untuk menampilkan grafik perkembangan fitness setiap generasi.

Selanjutnya terdapat data barang:

```
# Data barang: (nama, keuntungan, ukuran)
barang = [
    ("Barang1", 10, 5),
    ("Barang2", 40, 4),
    ("Barang3", 30, 6),
    ("Barang4", 50, 3),
    ("Barang5", 35, 7)
]
```

Data tersebut berisi nama barang, keuntungan, dan ukuran barang. Misalnya ("Barang1", 10, 5) berarti Barang1 memiliki keuntungan 10 dan ukuran 5.

Kemudian ditentukan kapasitas maksimal gudang:

```
ukuran_maksimal = 15 # Ukuran maksimal gudang
```

Artinya total ukuran barang yang dipilih tidak boleh melebihi 15.

Fungsi pertama adalah inisialisasi_populasi().

```
# INISIALISASI POPULASI
```

```
def inisialisasi_populasi(jumlah_populasi, jumlah_gen):
    populasi = []
    for i in range(jumlah_populasi):
        kromosom = [random.randint(0, 1) for _ in range(jumlah_gen)]
        populasi.append(kromosom)
    return populasi
```

Fungsi ini digunakan untuk membuat populasi awal secara acak. Setiap individu atau kromosom terdiri dari angka 0 dan 1. Nilai 1 berarti barang dipilih, sedangkan 0 berarti barang tidak dipilih.

Contoh kromosom:

[1,0,1,1,0]

Artinya Barang1, Barang3, dan Barang4 dipilih.

Fungsi berikutnya adalah hitung_fitness().

```
# EVALUASI FITNESS
def hitung_fitness(kromosom, barang, ukuran_maksimal):
    total_keuntungan = 0
    total_ukuran = 0
    for i in range(len(kromosom)):
        if kromosom[i] == 1:
            total_keuntungan += barang[i][1]
            total_ukuran += barang[i][2]
    if total_ukuran > ukuran_maksimal:
        return 0 # Penalti jika melebihi ukuran maksimal
    else:
        return total_keuntungan
```

Fungsi ini digunakan untuk menghitung nilai fitness dari setiap kromosom. Fitness dihitung berdasarkan total keuntungan barang yang dipilih.

Jika total ukuran barang melebihi kapasitas maksimal gudang, maka fitness akan bernilai 0 sebagai penalti.

Contohnya:

[1,1,0,1,0]

berarti memilih Barang1, Barang2, dan Barang4.

Total keuntungan:

- $10 + 40 + 50 = 100$

Total ukuran:

- $5 + 4 + 3 = 12$

Karena ukuran 12 masih di bawah 15, maka fitness bernilai 100.

Fungsi `roulette_wheel_selection()` digunakan untuk proses seleksi parent menggunakan metode Roulette Wheel Selection (RWS).

```
# SELEKSI: RWS (Roulette Wheel Selection)
def roulette_wheel_selection(populasi, fitness_populasi):
    total_fitness = sum(fitness_populasi)
    if total_fitness == 0:
        idx = random.randrange(len(populasi))
        return populasi[idx], idx
    probabilitas = [fitness / total_fitness for fitness in fitness_populasi]
    kumulatif_prob = []
    kumulatif = 0
    for p in probabilitas:
        kumulatif += p
        kumulatif_prob.append(kumulatif)
    r = random.random()
    for i, kum_prob in enumerate(kumulatif_prob):
        if r <= kum_prob:
            return populasi[i], i
    return populasi[-1], len(populasi) - 1
```

Metode ini bekerja seperti roda roulette. Individu dengan fitness lebih besar memiliki peluang lebih besar untuk terpilih sebagai parent.

Langkah pertama adalah menghitung total fitness seluruh populasi, kemudian menghitung probabilitas tiap individu. Setelah itu dipilih individu secara acak berdasarkan probabilitas tersebut.

Fungsi `uniform_crossover()` digunakan untuk proses crossover.

```
# Crossover: Uniform
def uniform_crossover(parent1, parent2):
    mask = [random.randint(0, 1) for _ in range(len(parent1))]
    anak1 = []
    anak2 = []
    for i in range(len(parent1)):
        if mask[i] == 0:
            anak1.append(parent1[i])
            anak2.append(parent2[i])
        else:
            anak1.append(parent2[i])
            anak2.append(parent1[i])
```

```
return anak1, anak2
```

Pada metode Uniform Crossover, setiap gen dibandingkan menggunakan mask acak bernilai 0 atau 1.

Jika mask bernilai 0 maka gen diambil dari parent pertama, sedangkan jika mask bernilai 1 maka gen diambil dari parent kedua.

Tujuan crossover adalah menghasilkan kombinasi kromosom baru yang lebih baik.

Fungsi `inversion_mutation()` digunakan untuk proses mutasi.

```
# MUTASI: Inversion
def inversion_mutation(kromosom):
    kromosom = list(kromosom)
    posisi1 = random.randint(0, len(kromosom) - 2)
    posisi2 = random.randint(posisi1 + 1, len(kromosom) - 1)
    kromosom[posisi1:posisi2] = list(reversed(kromosom[posisi1:posisi2]))
    return kromosom
```

Mutasi dilakukan dengan memilih dua posisi secara acak lalu membalik urutan gen di antara posisi tersebut.

Contoh:

[1,0,1,1,0]

dapat berubah menjadi:

[1,1,1,0,0]

Mutasi bertujuan untuk menambah variasi populasi agar algoritma tidak terjebak pada solusi tertentu saja.

Fungsi utama program adalah `run_ga()`.

```
# MAIN GA
def run_ga(jumlah_generasi, jumlah_populasi, prob_crossover, prob_mutasi,
ukuran_maksimal):
    jumlah_gen = len(barang)

    # Inisialisasi populasi awal
    populasi = inisialisasi_populasi(jumlah_populasi, jumlah_gen)

    best_fitness_list = []
    worst_fitness_list = []
    avg_fitness_list = []
```

```

all_fitness = []

best_individu = None
best_fitness_overall = 0

for generasi in range(jumlah_generasi):
    # Evaluasi fitness
    fitness_populasi = [hitung_fitness(individu, barang, ukuran_maksimal)
                        for individu in populasi]

    best_fitness = max(fitness_populasi)
    worst_fitness = min(fitness_populasi)
    avg_fitness = sum(fitness_populasi) / len(fitness_populasi)

    best_fitness_list.append(best_fitness)
    worst_fitness_list.append(worst_fitness)
    avg_fitness_list.append(avg_fitness)
    all_fitness.append(fitness_populasi.copy())

    if best_fitness > best_fitness_overall:
        best_fitness_overall = best_fitness
        index_best = fitness_populasi.index(best_fitness)
        best_individu = populasi[index_best]

    new_populasi = []
    used_indices = []

    while len(new_populasi) < jumlah_populasi:
        # Seleksi: RWS
        parent1, idx1 = roulette_wheel_selection(populasi,
fitness_populasi)
        used_indices.append(idx1)

        available_indices = [i for i in range(len(populasi)) if i not in
used_indices]
        if not available_indices:
            used_indices = [idx1]
            available_indices = [i for i in range(len(populasi)) if i !=
idx1]

        parent2, _ = roulette_wheel_selection(
            [populasi[i] for i in available_indices],
            [fitness_populasi[i] for i in available_indices]
        )
        used_indices.append(available_indices[_])

    # Crossover: Uniform
    if random.random() < probab_crossover:
        anak1, anak2 = uniform_crossover(parent1, parent2)

```

```

        else:
            anak1, anak2 = parent1[:,], parent2[:,]

        # Mutasi: Inversion
        if random.random() < probab_mutasl:
            anak1 = inversion_mutation(anak1)
        if random.random() < probab_mutasl:
            anak2 = inversion_mutation(anak2)

        new_populasi.extend([anak1, anak2])

    populasi = new_populasi[:jumlah_populasi]

# Grafik
plt.figure(figsize=(12, 7))

for i in range(jumlah_generasi):
    x = [i + 1] * len(all_fitness[i])
    y = all_fitness[i]
    plt.scatter(x, y, color='gray', alpha=0.1)

    plt.plot(range(1, jumlah_generasi + 1), best_fitness_list, color='blue',
label='Fitness Tertinggi')
    plt.plot(range(1, jumlah_generasi + 1), worst_fitness_list,
color='yellow', label='Fitness Terendah')
    plt.plot(range(1, jumlah_generasi + 1), avg_fitness_list, color='red',
label='Fitness Rata-rata')

plt.title('Perkembangan Nilai Fitness')
plt.xlabel('Generasi')
plt.ylabel('Nilai Fitness')
plt.legend()
plt.grid(True)
plt.show()

# Hasil akhir
selected_items = [barang[i][0] for i in range(len(best_individu)) if
best_individu[i] == 1]
selected_value = hitung_fitness(best_individu, barang, ukuran_maksimal)
selected_size = sum([barang[i][2] for i in range(len(best_individu)) if
best_individu[i] == 1])

print(f"Nilai Fitness Terbaik: {selected_value}")
print(f"Total Ukuran: {selected_size}")
print("Barang Terpilih:")
for item in selected_items:
    print(f"- {item}")

```

Fungsi ini menjalankan seluruh proses algoritma genetika mulai dari:

- inisialisasi populasi,
- evaluasi fitness,
- seleksi,
- crossover,
- mutasi,
- hingga pembentukan populasi baru.

Proses tersebut dilakukan berulang sebanyak jumlah generasi yang ditentukan.

Pada setiap generasi program menghitung:

- fitness tertinggi,
- fitness terendah,
- dan fitness rata-rata.

Nilai tersebut disimpan untuk ditampilkan dalam bentuk grafik perkembangan fitness.

Grafik digunakan untuk melihat apakah algoritma mengalami peningkatan solusi dari generasi ke generasi.

Di akhir program ditampilkan hasil terbaik berupa:

- nilai fitness terbaik,
- total ukuran barang,
- dan daftar barang yang dipilih.

Bagian berikut digunakan untuk menjalankan algoritma genetika:

```
# Menjalankan GA
run_ga(
    jumlah_generasi=50,
    jumlah_populasi=20,
    prob_crossover=0.5,
    prob_mutasi=0.1,
    ukuran_maksimal=15
)
```

Artinya program dijalankan dengan:

- 50 generasi,
- 20 individu dalam populasi,
- probabilitas crossover 50%,
- probabilitas mutasi 10%,
- dan kapasitas maksimal gudang sebesar 15.

HASIL OUTPUT

